

Staging Test Checklist

Build: b9cc9616 (Jul 28) **Listener:** <https://tscopier-worker-staging.up.railway.app> **Supabase:** axdcledcyhyvzrnfkwat **Frontend:** <https://staging.tscopier.ai>

READ THIS FIRST — Staging limitations vs production

Capability	Production	Staging now	What's needed
Telegram connections	53 real sessions	3 real + 59 blank	—
Live signal flow	~1000/day per user	0 (need to post test signals)	Post signals in SIGNALS TESTER channel
Broker execution	FxSocket live	No broker configured	Staging FxSocket account + broker credentials
Signal formats tested	All variants	None	15+ test signals in SIGNALS TESTER channel
Scale load	53 sessions	62 entries	Already done (synthetic)
Failure scenarios	Random (real)	None injected	Manual injection needed
Automated assertions	None	None	Log queries + alert rules

Tests marked with **require setup first** (test channel, broker account). Tests marked with **can run right now**.

0. PREREQUISITES — Set up the test environment

Before ANY end-to-end testing can happen, we need:

0.1 The 3 real staging bots (all Nigerian +234 numbers)

#	User ID	Phone	Created
1	f1d54bc2-d6d4-4c4e-be94-13b11bc7d049	+2348139186861	Jul 23
2	ed0ab337-af39-4e94-a7d0-49d00542de31	+2348142415769	Jul 23
3	4d2c9a06-0443-458f-94bb-8cd69b64d968	+2349054538604	Jul 24

0.2 Test channel (already exists)

- **Channel ID:** -1003962048504
- **Display name:** SIGNALS TESTER
- **All 3 staging bots are members**

0.3 Build the test signal library

Prepare messages to post in the test channel. Every format below must be covered.

0.4 Set up log queries

Bookmark these Supabase queries for quick access during testing:

```
-- All listener events in last hour
SELECT * FROM listener_events WHERE created_at > now() - interval
'1 hour' ORDER BY created_at DESC;
```

```
-- All signals in last hour
SELECT id, user_id, channel_id, parsed_data, created_at FROM
signals WHERE created_at > now() - interval '1 hour';
```

```
-- All errors in last hour
SELECT * FROM listener_events WHERE event_type LIKE '%error%' OR
event_type LIKE '%fail%' ORDER BY created_at DESC;
```

0.5 Configure staging broker accounts (for trade execution tests)

Required: Staging FxSocket API key + a demo MT4/MT5 account to execute test trades.

1. INFRASTRUCTURE & STARTUP

1.1 All services boot correctly

☐

Listener health: `curl https://tscopier-worker-staging.up.railway.app/health` returns `{"ok":true}`

☐

Frontend loads: `https://staging.tscopier.ai` renders dashboard without JS errors

☐

Supabase reachable: Staging Supabase dashboard loads, queries return data

1.2 All network connections established

Check startup logs for each connection type:

☐

Supabase REST: Log shows Supabase queries succeeding

☐

Supabase Realtime (telegram_channels): `[sessionManager] Realtime telegram_channels subscription active`

☐

Supabase Realtime (auth_pending): `[sessionManager] Realtime telegram_auth_pending subscription active`

☐

Telegram MTProto (3 real sessions): `[sessionManager] Started listener for user-<id> for all 3`

☐

Telegram MTProto (59 synthetic): `[sessionManager] Failed to start listener for user-<id> for all 59 (graceful)`

☐

HTTP server: Listener HTTP routes responding on `WORKER_PORT`

1.3 All timers and intervals running (62 sessions)

Timer	Expected count	Period	Check method
Watchdog	3 (real sessions only)	30s	Log [userListener] watchdog_trigger (on failure)
Safety poll	3	10s	Log [userListener] polled N channels
Fast poll	3	3s	Log [userListener] fastPoll
Session persist	3	30min	Log [userListener] session_persisted
Reply chain sweep	3	60s	Log [userListener] reply_chain_sweep
Signal reconcile sweep	3	30s	Log [userListener] signal_reconcile
Entity warmup	3	10min	Log [userListener] entity_warmup
syncSessions	1	30s	[sessionManager] syncSessions
renewAllLeases	1	10-20s	[sessionManager] renewAllLeases
Realtime health check	1	60s	Log on failure only
Auth cleanup	1	60s	[authService] cleanup

☐

All 3 real listeners have active watchdog, safety poll, fast poll

☐

syncSessions runs every 30s without error

☐

renewAllLeases completes for all 62 sessions without error

☐

No renewLeasesInFlight skip messages (means a cycle is wedged)

1.4 Monitor loops active

Check TradeExecutor monitor loops:

☐

TradeExecutor sweep loop: every 1s (active) / 15s (idle)

☐

VirtualPendingMonitor: every 200ms (active) / 15s (idle)

☐

RangeBrokerPendingMonitor: every 2s (active) / 15s (idle)

☐

SignalEntryPendingMonitor: every 2s (active) / 15s (idle)

☐

SignalRangeEntryMonitor: every 2s (active) / 15s (idle)

☐

PartialTpMonitor: every 2s (active) / 15s (idle)

☐

OpenTradeReconcileMonitor: every 30s (active) / 120s (idle)

☐

TrailingStopMonitor: every 400ms (active) / 15s (idle)

☐

AutoManagementMonitor

☐

BasketSLTpReconcileMonitor: every 5s (active) / 15s (idle)

☐

CopyLimitMonitor: every 30s

☐

NewsTradingMonitor: every 60s

☐

ChannelReconcileMonitor: every 45s

☐

ChannelListenerManager periodic sync

1.5 Queue system (if enabled)

☐

Redis streams reachable: no RedisStreamsError in logs

☐

Consumer manager started (trade worker side)

☐

Publisher writes without error (listener side)

2. END-TO-END SIGNAL PIPELINE

Requires: user posts test signals to SIGNALS TESTER channel.

2.1 Standard entry signal — full lifecycle

Action: Post this in the test Telegram channel:

BUY XAUUSD 1.00 SL 3980 TP 4000

Expected pipeline (trace each stage in logs):

Step	Log marker	Time budget
Telegram receives	GramJS MTProto message	< 2s
NewMessage handler	[userListener] onNewMessage	< 100ms
Parse (deterministic)	[parseSignal] parsed entry	< 500ms
Signal row created	[ensureSignalRow] ensured signal <uuid>	< 100ms
Pipeline timestamp	[pipeline] pipeline_ts with keys	< 50ms
Route to dispatch	routeDispatchToTradeWorker	< 100ms
Queue push (or HTTP)	enqueueParsedSignal or pushParsedSignal	< 1s
TradeExecutor receives	[dispatch] dispatching signal	< 2s
Broker order attempt	[orderSend] or error log	< 5s

☐

Signal appears in signals table (check with `SELECT * FROM signals WHERE channel_id = '<test-channel>'`)

☐

Parsed data contains: side=buy, symbol=XAUUSD, lots=1.00, sl=3980, tp=4000

☐

pipeline_ts has timestamps for telegram_received_at, parsed_at, ensured_at, dispatched_at

2.2 Entry with zone

SELL XAUUSD 3950-3960 SL 3980 TP 3930 TP 3910

☐

Parse produces range entry with entry_min=3950, entry_max=3960

☐

SL at 3980 (above highest TP for sell — validates sanitizeBasketTargetStops)

☐

TPs parsed as ladder: [3930, 3910]

2.3 Entry with SL inside TP ladder (invalid)

BUY XAUUSD 1.00 SL 4020 TP 4030 TP 4050

☐

sanitizeBasketTargetStops rejects SL=4020 as inside TP ladder (4030-4050)

☐

Log shows incoherent_sl_within_tp_ladder

2.4 No-stop entry (naked signal)

BUY XAUUSD 1.00

☐

Parsed as entry without SL/TP

☐

Signal row created with null stops

☐

No stripInvalidStopsForSide error (nothing to strip)

2.5 Dot-leader format (Martin's new format)

Sell.....4080
Sl.....4090
Tp.....4071

☐

Parsed as entry: side=sell, price=4080, sl=4090, tp=4071

☐

From Martin's commit ebed3860 dot-leader regex

2.6 Management — Adjust SL

Adjust SL to 3940

☐

aiParseModification or deterministic parse identifies as adjust_sl

☐

Produces signal with intent=modify, sl=3940

☐

No entry side needed (modifies existing basket)

2.7 Management — Breakeven

BE at 3970

☐

Parsed as breakeven management intent

☐

autoManagement system triggers BE logic when price hits 3970

2.8 Management — Close

CLOSE XAUUSD at 4010

☐

Parsed as close intent

☐

Route to broker close path

☐

orderCloseAudit logs the close (Martin's commit 6274eb78)

2.9 Close worse entries

Close worse entries

☐

Parsed as close_worse_entries

☐

Only closes legs worse than current price, not the full basket

2.10 Edit existing message (signal revision)

Action: Post any entry signal, then edit it within 30s.

☐

GramJS fires edit event

☐

tryApplyMessageRevision triggered

☐

AI re-parse (or deterministic fallback) produces revised signal

☐

Original signal updated, not duplicated

2.11 Signal with TP ladder (multi-TP)

BUY XAUUSD 1.00 SL 3980 TP 4000 / TP 4020 / TP 4040 / TP 4060

☐

TP ladder parsed: [4000, 4020, 4040, 4060]

☐

Partial TP monitor tracks each rung

☐

partialTpMonitor fires when each TP hit

☐

Trail stop triggers after partial TP fills

2.12 Underscore format (Martin's expandSymbolSideUnderscore)

XAUUSD_BUY 1.00 SL 3980 TP 4000

☐

expandSymbolSideUnderscore normalizes to XAUUSD BUY

☐

Parsed correctly with side=buy, symbol=XAUUSD

2.13 Daily bias / chatter (non-actionable)

BIAS: Bullish above 3950, bearish below

☐

Parsed as non-actionable

☐

No signal row created



No dispatch attempt



3. FAILURE INJECTION

3.1 AUTH_KEY_DUPLICATED recovery



Force restart: push to staging branch → Railway redeploys



Old worker logs `disconnect_start` → `disconnect_complete` for all 62 sessions



drain timeout $\geq$ 30s (not truncated)



New worker connects 3 real sessions cleanly



Zero AUTH_KEY_DUPLICATED errors in first 5 minutes



`releaseOwnedSessionLeases` runs, 0 unresolved leases remain



If AUTH_KEY_DUPLICATED occurs anyway: recovery fires (exponential backoff), max 10 retries, then session invalidation

3.2 BinaryReader crash guard



Patch `telegram+2.26.22.patch` verified applied (check `node_modules` markers)



If malformed RPC: `noteMalformedRpcResult` → reconnect in <5s



Rate-limited: max 10 recoveries per 10-min window



After exhaustion: session invalidated, not stuck in retry loop



Raw fallback catches `readUInt32LE / Cannot read properties of undefined`

3.3 CHANNEL_INVALID auto-disable



Remove bot from a monitored channel



Counter increments 1..5 in logs: [userListener]
channel_invalid_detected count=N/5



At 5: DB update telegram_channels.is_active = false



channel_auto_disabled event in listener_events



Listener stops polling that channel



Re-enable via UI → counter resets, polling resumes



USERNAME_NOT_OCCUPIED also counted (stale username)



TIMEOUT errors NOT counted (transient distinction)

3.4 TIMEOUT death spiral



Simulate connection timeout by blocking GramJS TCP



client.onError catches TIMEOUT → requestReconnect('update_loop_timeout')



Reconnect happens, listener resumes



No infinite TIMEOUT loop (was the old bug)

3.5 Realtime subscription drop



Wait for natural Supabase WebSocket drop (20-40 min) OR toggle realtime off/on



Log: Realtime telegram_channels subscription CHANNEL_ERROR – retrying in 5s



Within 5s: subscription active again



Health check timer (60s) never fires (5s retry is working)



On shutdown: health check timer stopped, no leaks

3.6 Lease renewal wedged

☐

renewLeasesInFlight flag prevents stacking

☐

If previous cycle hung, skip + log: renewAllLeases skipped

☐

Next cycle (10-20s later) retries normally

3.7 OpenAI 429 (rate limit)

☐

Simulate 429 from AI parsing endpoint

☐

REVISION_AI_RETRY_COOLDOWN_MS (10min) prevents retry spam

☐

Falls back to deterministic parse (no signal lost)

3.8 Flood wait

☐

Telegram FLOOD_WAIT error from GramJS

☐

tgInvoke transparently backs off (default max 2 retries, 90s)

☐

Operation retries after wait period

3.9 SyncSessions handles DB changes

☐

Directly update a synthetic session to is_active=false in Supabase

☐

Within 30s: syncSessions detects change, stops the listener, releases lease

☐

Re-enable it → syncSessions restarts it

3.10 Channel changes via Realtime

☐

Add a new channel to telegram_channels for a real user via Supabase

☐

Realtime subscription picks it up within 5s

☐

`listener.onChannelsChanged()` fires, reloads channels

4. SCALE & LOAD (4-hour test)

4.1 Startup load (62 sessions)

☐

All 62 sessions processed by `loadAll()` without throwing

☐

3 real sessions connect within 30s of startup

☐

59 synthetic sessions fail with graceful error (no crash)

☐

Total startup time < 120s (60s timeout × 62 sessions runs in parallel with concurrency)

4.2 Memory stability

☐

Memory stabilizes after 30 min warmup (flat line)

☐

No linear growth over 4 hours (no leak)

☐

No `MaxListenersExceededWarning` (timer overload)

☐

No heap OOM or FATAL ERROR

4.3 CPU stability

☐

CPU < 50% during normal operation

☐

CPU spikes only during syncSessions / lease renewal cycles

☐

No sustained 100% CPU

4.4 Event loop health

- ☐ No ETIMEDOUT from Supabase queries
- ☐ No connection pool exhaustion
- ☐ Lease renewal cycles complete for all 62 sessions (check `renewAllLeases` log)
- ☐ `syncSessions` cycle time stable (< 5s per cycle)

4.5 Channel polling at scale

- ☐ 170 channels polled across all sessions without backlog
 - ☐ No poll queue build-up (polls complete before next interval)
 - ☐ No `poll_peer_resolve_failed` for real channels (synthetic ones expected)
-

5. DATABASE & CRON

5.1 Migrations applied

- ☐ `20260728120000_purge_stale_trade_prices_cron` migration present
- ☐ All prior migrations present (127 total)
- ☐ No schema cache errors in logs

5.2 Purge cron

- ☐ `purge_stale_trade_prices` job registered in `cron.job`
- ☐ Runs every 5 min without error
- ☐ Active basket data NOT purged

☐

Flat baskets: ladder rows cleaned up within 5 min

☐

`channel_active_trade_params` kept for active channels

5.3 Data integrity

☐

No FK violations in logs (trades, `execution_logs` referencing signals)

☐

No orphaned signal rows

☐

No orphaned `range_pending_legs` for non-existent baskets

6. FRONTEND

Requires: `staging.tscofier.ai` accessible + logged-in staging user.

6.1 Dashboard loads

☐

Dashboard page renders without console errors

☐

All locale files load (check network tab for 200s on all `.json` files)

☐

No React hydration errors

6.2 CopierStatusCard (Martin's commit 70de046f)

☐

Card shows broker connections (healthy/issues/none)

☐

Copier engine status: live/offline/checking

☐

Telegram listener status: online/offline

☐

Summary string displays correctly (“All checks passed” / “Issues detected”)

☐

Collapsed/expanded state persists across page reload (`localStorage`)

6.3 ListenerLeaseOfflineBanner (Martin's commit cf31c7e8)

- ☐ Warning banner shows when lease expired
- ☐ Amber with AlertCircle icon
- ☐ "Reconnect Telegram" link navigates to /channels
- ☐ Banner hides when lease is active

6.4 Channel management

- ☐ Add channel page loads
- ☐ List channels shows connected channels
- ☐ Disabled channels show "channel disconnected" badge

6.5 Telegram auth flow

- ☐ Phone auth: send_code → verify_code completes
- ☐ QR auth: QR code displays, scan completes
- ☐ Auth errors show user-friendly messages (not raw API errors)
- ☐ Session re-link works after invalidation

6.6 i18n (all 6 languages)

- ☐ en, es, fr, ar, ja, nl, pl, ru, sv all render
 - ☐ No missing key errors in console
 - ☐ RTL layout for Arabic
-

7. SHUTDOWN & RESTART

7.1 Graceful shutdown



SIGTERM → shutdown() sequence:

1. HTTP server stops accepting requests
2. Auth service disconnects pending auth clients
3. Signal queue consumers stop
4. Trade executor stops (sweep loop, timers, Realtime channels)
5. All monitors stop
6. `sessionManager.disconnectAll()`: all 62 listeners stopped, all leases released
7. Drain wait (30s) for Telegram to close connections
8. `process.exit(0)`

7.2 Restart recovery



After shutdown → startup: all 62 sessions reload



No orphaned leases from previous run



No AUTH_KEY_DUPLICATED (drain prevented race)

7.3 Shutdown during reconnect



Force reconnect in progress, then send SIGTERM



stopping flag prevents new reconnect cycles



shutdown completes, no stray async operations

8. SESSIONS & AUTH

8.1 Session persistence



Session string persists across restart (check: listener reconnects with same session)

☐

sessionPersistTimer (30min) saves session string if changed

☐

After 30 min: no auth prompt, still connected

8.2 AUTH_KEY_UNREGISTERED recovery

☐

If session is invalidated: invalidateTelegramSession() called

☐

Session string deleted from telegram_sessions

☐

Auth pending row cleared

☐

Lease released

☐

User sees “re-link Telegram” in UI

8.3 Max retry exhaustion

☐

AUTH_KEY_DUPLICATED retries exhausted (10 cycles) → session invalidated

☐

Not stuck in infinite retry (was the old bug)

9. ALL MARTIN’S COMMITS — SPECIFIC CHECKS

9.1 Basket layering: flat basket purge (commits 03d21caf, 5ed2571c)

Requires test trade execution: - [] Open a basket (send entry signal) → ladder rows created - [] Close all trades → shouldLockBasketLayering returns basket_fully_closed - [] Ladder rows purged immediately - [] Layer-till-close ON does NOT prevent purge on flat baskets - [] Pre-claim stale check: [fireLeg] stale basket – skipping leg claim - [] Send another signal → new basket starts fresh (no stale rows)

9.2 Trade reconciliation: empty OpenedOrders guard (commit 5dd36c5b)

☐

Code review confirms mass ghost-close is deferred on empty OpenedOrders

☐

stripInvalidStopsForSide does not strip legitimate near-market stops

9.3 ensureSignalRow (commit 186c8d1c)

☐

Signal row exists before FK-dependent writes

☐

No FK violation errors in `trade_execution_logs` or `trades`

☐

Retry loop on FK violation fires and recovers

9.4 Order close audit (commit 6274eb78)

☐

[`orderCloseAudit`] logs on every close operation

☐

Stack trace included

☐

Both `fxClient` (v1) and `fxsocketClient` (v2) covered

9.5 Purge cron (commit 70de046f migration)

☐

Cron registered

☐

Active data preserved

☐

Flat data cleaned within 5 min

9.6 SL/TP validation (commit ebed3860)

☐

SL inside TP ladder → rejected

☐

- ☐ Directionally invalid SL/TP → stripped
- ☐ Dot-leader format → parsed
- ☐ Underscore format → parsed

9.7 Frontend components (commits cf31c7e8, 70de046f)

- ☐ ListenerLeaseOfflineBanner visible when offline
- ☐ CopierStatusCard renders all status indicators
- ☐ All 6+ locale files load

9.8 Pipeline observability (commit 03dffd53)

- ☐ Pipeline events emitted for each signal stage
 - ☐ setPipelineTimestamp calls visible in logs
 - ☐ 20+ timestamp keys populated across the pipeline
-

10. PRODUCTION ROLLOUT SMOKE TEST

10.1 Pre-merge checklist

- ☐ All critical tests above pass
- ☐ Staging listener has been running for 4+ hours without issues
- ☐ No ERROR or FATAL in staging logs
- ☐ All 3 real sessions connected for entire 4-hour window
- ☐ Memory stable (no leak)
- ☐ All 13 Martin's commit checks pass

10.2 Post-merge monitoring (2 hours)

☐

git push upstream staging:main → Railway deploys

☐

Zero TIMEOUT death spirals

☐

Zero QR login AUTH_KEY_UNREGISTERED loops

☐

Zero BinaryReader crashes

☐

Zero CHANNEL_INVALID infinite retry loops

☐

Zero AUTH_KEY_DUPLICATED from deploy race (drain worked)

☐

All 53 production sessions reconnect within 60s

☐

No signal delivery regression (compare per-channel latency pre/post)

☐

No unexpected lease expiry (53 leases all active)

☐

No Realtime subscription drops >5s

10.3 Rollback plan

If any critical failure appears in first hour:

```
git push upstream main~1:main --force
# Railway auto-deploys previous version
```

Time to rollback: ~2 minutes.

QUICK SUMMARY: What to run daily

When	What to run	Time
Every deploy	1.1-1.3 (startup health), 3.1 (AUTH_KEY_DUP), 3.5 (Realtime reconnect)	15 min
Daily	2.1-2.3 (entry signals), 2.6-2.8	30 min

When	What to run	Time
	(management), 3.3 (CHANNEL_INVALID)	
Weekly	Full Section 2 (all signal formats), Section 4 (scale load 4h)	4.5 hours
Before prod rollout	Everything above	1 day

TEST TRACKING

Copy this section into your test run document:

Test date: _____ **Build:** _____ **Tester:** _____

Pass/Fail summary

Section	Total	Pass	Fail	Blocked
1. Infrastructure & Startup	4	–	–	–
2. E2E Signal Pipeline	13	–	–	–
3. Failure Injection	10	–	–	–
4. Scale & Load	5	–	–	–
5. Database & Cron	3	–	–	–
6. Frontend	6	–	–	–
7. Shutdown & Restart	3	–	–	–
8. Sessions & Auth	3	–	–	–
9. Martin's Commits	8	–	–	–
10. Production Smoke	3	–	–	–

Blocked by: (list what setup is missing)